

LLMOrchestrator

خلاصہ

برای ایجاد، مدیریت و ارتباط Go یک ماژول مستقل و قابل استفاده مجدد LLMOrchestrator از طریق پروتکل ترکیبی لوله+فایل است که شامل قطع‌کننده‌های مدار برای هر (Qwen Code، Junie، Gemini، Claude Code، OpenCode) CLI با عوامل بدون رابط کاربری جداشده و تضمین‌های تست ضد بلوف i18n عامل، انتخاب چندرأئیدهنده قابل اتصال، انتزاع می‌شود.

توضیح کوتاه

CLI که یک رابط یکپارچه برای ایجاد و هدایت چندین عامل Go یک ماژول قابل استفاده مجدد از طریق پروتکل ترکیبی لوله+فایل فراهم می‌کند. استخرسازی امن عوامل با LLM مبتنی بر i18n قطع‌کننده‌های مدار و راهبردهای مسیریابی قابل انتخاب، عمداً مستقل از مصرف‌کننده، با مترجم قابل اتصال.

توضیح مفصل

زیرساخت اشتراکی برای هماهنگ‌سازی عوامل کدنویسی بدون رابط کاربری LLMOrchestrator است — همان لوله‌کشی که هر سیستم چندعاملی به آن نیاز دارد و معمولاً به شکلی ناقص CLI بازطراحی می‌شود. به جای آنکه هر پروژه مجبور باشد ایجاد فرایند، قالب‌بندی پیام و تجزیه نتایج را OpenCode، Claude Code، Gemini CLI، Junie و Qwen و AgentPool یک استخر امن نخ، Agent از نو پیاده‌سازی کند، این ماژول یک رابط یکپارچه Code ارائه می‌دهد که عوامل را از چندین ارائه‌دهنده پشت یک نمای واحد MultiProviderPool یک قابل اتصال است — نوبت‌دهی دایره‌ای AgentSelector گردآوری می‌کند. مسیریابی از طریق یک که ارائه‌دهندگان ناتوان در برآورده کردن الزامات را نادیده می‌گیرد، یا اولویت‌بندی با جایگزین — به طوری که نحوه توزیع کار یک سیاست انتخابی شماسست، نه یک فرض کدگذاری شده. هر عامل مشخص مشترک است که مالک چرخه کامل فرایند است: شروع BaseAdapter یک آداپتور نازک بر روی یک — راه‌اندازی مجدد و زنده‌بودن، SIGKILL و سپس SIGTERM با راه‌اندازی لوله، توقف آرام با همان بخش پیچیده و مستعد خطا که یک بار برای همیشه حل شده است.

با JSON ارتباط عمده ترکیبی است و انتقال را با وظیفه منطبق می‌کند. انتقال لوله‌ای حامل پیام‌های جداکننده خط، با مهلت خواندن برای هر درخواست و سقف طول پاسخ برای پیام‌رسانی تعاملی سریع است، در حالی که انتقال فایلی از دایرکتوری‌های ورودی/خروجی/مشترک برای هر جلسه برای مصنوعات بزرگ یا ماندگار که نباید در لوله قرار گیرند استفاده می‌کند. تاب‌آوری یک فکر بعدی نیست — بلکه ساختاری است: قطع‌کننده مدار برای هر عامل پس از سه شکست متوالی به مدت ۶۰ ثانیه باز می‌شود و سپس یک پروب نیمه‌باز انجام می‌دهد، و یک مانیتور سلامت پس‌زمینه عوامل را پینگ می‌کند تا عاملی که از کار افتاده بتواند بدون انتظار برای ترافیک ورودی بهبود یابد. دریافت استخر روی یک متغیر را در انتظار مشغول نگه دارد، و تجزیه‌کننده پاسخ بدون CPU شرطی بلاک می‌شود به جای آنکه حالت و امن برای فراخوانی همزمان است. این ماژول کاملاً جدا شده است — هیچ جزئیات i18n مصرف‌کننده‌ای اجزای ورود ندارد — و هر رشته قابل مشاهده توسط کاربر از طریق یک مترجم که شناسه‌های پیام را به صورت NoopTranslator عبور می‌کند، با یک Translator قابل اتصال تحت‌اللفظی برمی‌گرداند تا ترجمه گم‌شده به وضوح نمایان شود نه پنهان.

چرا آن را ساختیم

را به شکلی قابل اعتماد راه‌اندازی و با آنها ارتباط برقرار CLI هر سیستم چندعاملی نیاز دارد عوامل کند. حل مجدد مسائل ایجاد فرایند، قالب‌بندی، تجزیه و مدیریت شکست برای هر پروژه اتلاف وقت و این موارد را در یک ماژول جدا شده و قابل استفاده مجدد LLMOrchestrator. مستعد خطاست متمرکز می‌کند که مسئولیت تخصصی آن، قابلیت استفاده مجدد را تضمین می‌کند — و این قابلیت استفاده مجدد به محض ورود هر جزئیات مصرف‌کننده‌ای از بین می‌رود.

محتوا

چرا یک تغییردهنده بازی است

از یک کار مهندسی سفارشی و «CLI این ابزار، تبدیل کردن «اجرای گروهی از عامل‌های ناهمگن، طاقت‌فرسا برای هر پروژه به یک ایمپورت ساده کتابخانه‌ای را ممکن می‌سازد — مدیریت تجمیع شکست مدار، چرخه حیات و مسیریابی قابل تعویض، همگی از پیش حل شده و مقاوم‌سازی شده‌اند، و چون تست‌های ضد-فریب آن، سیستم واقعی را سرتاسر آزمایش می‌کنند و به «کامپایل می‌شود» بسنده نمی‌کنند، به انتزاعی دست می‌یابید که واقعاً می‌توانید در شرایط همزمانی و شکست به آن اعتماد کنید. نه چیزی که فقط در نمودار خوب به نظر برسد.

نوآوری‌ها

- **stdin/stdout روی JSON سرعت تعاملی (خطوط) — پروتکل ترکیبی لوله+فایل.**
/تبادل پایدار مبتنی بر فایل (صندوق ورودی/خروجی و (مهلت‌های خواندن، محدودیت پاسخ (مشترک) برای مصنوعات بزرگ، به‌طوری که هرگز مجبور نیستید تأخیر را به پای حوام قربانی کنید یا برعکس.
- **یک نمای واحد روی چندین — تجمیع چند ارائه‌دهنده قابل تعویض با انتخابگرهای قابل تعریف**
با مسیریابی نوبتی یا اولویت‌محور که به‌عنوان سیاست انتخاب می‌شود نه CLI ارائه‌دهنده به‌صورت ثابت.
- **و کاهش خودکار کلایی — مدار شکن انفرادی برای هر عامل + پایشگر سلامت پس‌زمینه‌ای**
بازیابی (۳ شکست → ۶۰ ثانیه قطع → کاوش نیمه‌باز)، به‌طوری که یک عامل ناپایدار ایزوله و سپس بدون دخالت دستی به‌آرامی بازگردانده می‌شود.
- **بلاک می‌شود تازمانی که یک عامل sync.Cond روی Acquire — تجمیع بدون انتظار فعال**
سالم و منطبق آزاد شود یا زمینه درخواست لغو گردد، بنابراین انتظار هیچ بار پردازشی ندارد.

- شناسه‌های پیام را NoopTranslator — جداسازی دقیق + ضد-فریب بین‌المللی‌سازی به صورت خام برمی‌گرداند تا نبود ترجمه هرگز نادیده گرفته نشود، نه اینکه به صورت خاموش خالی بماند.
- فهرست سفید مسیرهای باینری به این معنی است که هیچ‌گونه — امنیت به صورت پیش فرض درون‌یابی شل و در نتیجه هیچ سطحی برای تزریق فرمان وجود ندارد، همراه با محافظت در برابر پیمایش مسیر، محدودیت پاسخ ۱ مگابایتی برای جلوگیری از خروجی‌های کنترل‌نشده، و در لاگ‌ها API ماسک کردن کلیدهای
- JSON/دورهای رفت و برگشت واقعی دیسک — **Challenge** چارچوب تست ضد-فریب پارسر در پنج زبان، با یک دروژه جهش همزمان که باید در صورت خرابی ویژگی، با کد خروج غیرصفر پایان یابد — تستی که ثابت می‌کند واقعاً می‌تواند شکست بخورد.

بزرگ‌ترین چالش‌های فنی و راه‌حل‌های ما

- ایجاد شده به طرز فریبنده‌ای CLI ارتباط با یک فرایند ورودی/خروجی قابل اعتماد فرایند عامل دشوار است؛ با یک حمل و نقل ترکیبی لوله+فایل، قرارداد پیام/پارسر تعریف شده تا هر دو طرف بر که کل چرخه حیات فرایند از جمله مهلت BaseAdapter قالب سیم توافق داشته باشند، و یک را متمرکز می‌کند، حل شد SIGKILL با بازگشت به SIGTERM منظم
- متشکل از متغیر شرطی و قفل که در آن AgentPool با یک همزمانی بدون انتظار فعال تازمانی که یک عامل منطبق و سالم واقعاً آزاد شود یا زمینه درخواست لغو گردد، به Acquire خواب می‌رود، همراه با یک پارسر بدون حالت و عاری از اثر جانبی که ایمن است از چندین گوروتین به طور همزمان فراخوانی شود، حل شد
- به گونه‌ای حل شد که یک ارائه‌دهنده معیوب نتواند بقیه را با خود. ایزوله‌سازی شکست ارائه‌دهنده پایین بکشد: مدارشکن‌های انفرادی برای هر عامل شعاع انفجار را محدود می‌کنند، و یک گوروتین پایشگر سلامت، بازیابی را حتی زمانی که هیچ درخواستی برای تحریک آن نمی‌رسد، هدایت می‌کند.
- en/ حل شد: ده‌ها ثابت در Challenge با یک اجراکننده اثبات درستی، نه فقط کامپایل که سیستم واقعی را آزمایش می‌کنند، به علاوه یک دروژه جهش همزمان sr/ja/es/de باید شکست بخورد → خروجی بسته‌بندی با کد ۹۹ که (LLMORCH_MUTATE_RUNNER=1) عمداً ویژگی را خراب می‌کند تا ثابت کند خود دروژه فریب نیست
- با شناسه‌های خام و تزریق NoopTranslator با درز بومی‌سازی بدون شکست خاموش مترجم به زای هر مصرف‌کننده حل شد، به طوری که نبود ترجمه همیشه قابل مشاهده باشد و نه اینکه پنهان شود.

محتوا

پشته فناوری

- به خاطر پشتیبانی درجه یک از همزمانی و کنترل دقیق پردازش‌ها انتخاب — **Go (نسخه ۱.۲۵)** شده است؛ دقیقاً همان چیزی که هماهنگی فرایندهای زنده‌ی عامل‌ها نیاز دارد. این ابزار ماژول آداپتورهای عامل، ترنسپورت‌ها و پارسر را پیاده‌سازی می‌کند.
- انتخابی آگاهانه برای کمینه — **yaml.v3 و testify (به علاوه Go کتابخانه استاندارد)** تا ماژول سبک‌وزن LLM‌های SDK نگه‌داشتن سطح وابستگی‌ها و عدم استفاده از هیچ‌یک از باقی بماند و بتواند بدون کشیدن بار اضافی فروشنده، در هر مصرف‌کننده‌ای جاسازی شود.
- برای پیام‌رسانی تعاملی سریع انتخاب — **(std.io JSON-lines) ترنسپورت پایپ** شده است که با تعیین مهلت‌های خواندن و محدودیت‌های طول پاسخ تقویت شده تا یک عامل معلق یا سرکش نتواند فراخواننده را مسدود کند.

- برای تبادل مصنوعات بزرگ و پایدار در هر — ترنسپورت فایل (صندوق ورودی/خروجی/مشترک) . جلسه انتخاب شده است، جایی که پایپ ابزار مناسبی نیست.
- **sync.Mutex/sync.Cond** — برای پیاده‌سازی دریافت عادلانه و مسدودکننده استخر عامل‌ها بدون انتظار فعال انتخاب شده‌اند.
- به‌طور مشترک برای ایجاد تاب‌آوری در سطح **HealthMonitor + مدار قطع‌کننده** . عامل و بازیابی فعال انتخاب شده‌اند، نه فقط تشخیص شکست.
- به‌عنوان درز محلی‌سازی جداشده انتخاب شده است تا رشته‌های خاص — **pkg/i18n مترجم** . مصرف‌کننده از هسته جدا بمانند.
- **(challenges/runner) + Makefile (test -race, fuzz, cover)** چارچوب چالش برای تأیید مبتنی بر شواهد و ضدحقه انتخاب شده‌اند، از جمله تشخیص شرایط رقابتی و — . فازی‌کردن پارسر تا درستی تحت شرایط خصمانه اثبات شود، نه اینکه صرفاً فرض شود.

وضعیت و یادداشت‌های شفافیت

- یک ماژول قابل‌استفاده مجدد و جداشده که به‌عنوان زیرماژول در چندین پروژه از **وضعیت: بتا** . عمومی است **GitHub** ؛ مخزن **Apache-2.0** :مجوز . مصرف می‌شود **Helix/vasic** .
- پل زده می‌شود؛ این ماژول به‌طور **LLMsVerifier** از **HelixQA** ابرداده‌های مدل از طریق **LLMsVerifier/VisionEngine/DocProcessor** مستقیم را وارد نمی‌کند. پشته‌های **helix_code** به (و غیره **Gin/PostgreSQL**) اپلیکیشن والد **CLAUDE.md** ذکر شده در **helix_code** اشاره دارند، نه این ماژول.

(ماژول قابل‌استفاده مجدد و جداشده — **LLM** خوشه زیرساخت) اصلی-**Helix** :سطح اولویت . قرار دارد **HelixTrack** پس از